HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CNTT 1

BỘ MÔN THỰC TẬP CƠ SỞ

BÁO CÁO TIẾN ĐỘ TUẦN

HỆ THỐNG CHAT REALTIME TÍCH HỢP AI PHÂN LOẠI NỘI DUNG ĐỘC HẠI

Giảng viên	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Phạm Nhật Minh**
Mã sinh viên	**: B23DCCE066**
Lớp	**: D23CQCE06-B**

Hà Nội – 2026

I. Mục tiêu công việc

- Thiết lập nền tảng bảo mật: Xây dựng các hệ thống như Đăng ký, Đăng nhập và Đăng xuất cho ứng dụng.
- Quản lý phiên làm việc (Session): Sử dụng cơ chế JWT (Access Token) và Refresh Token để duy trì trạng thái đăng nhập mà không cần người dùng nhập mật khẩu liên tục.
- Bảo vệ tài nguyên: Tạo Middleware để kiểm soát quyền truy cập, đảm bảo chỉ người dùng đã xác thực mới có thể sử dụng các API nội bộ.
- Kết nối dữ liệu: Thiết lập kết nối ổn định với cơ sở dữ liệu MongoDB để lưu trữ thông tin người dùng và phiên làm việc.

II. Nội dung đã hoàn thành.

1. Controller Xác thực (controllers/authController.js)

Đây là phần điều khiển logic của việc xác thực bao gồm:

- bcrypt: Thư viện dùng để băm (hash) mật khẩu, giúp bảo mật thông tin người dùng.
- jwt: Dùng để tạo mã thông báo (Access Token) định danh người dùng.
- ACCESS_TOKEN_TTL = '30m': Thời gian sống của Access Token là 30 phút.
- REFRESH_TOKEN_TTL: Thời gian sống của Refresh Token (14 ngày).

Hàm signUp (Đăng ký):

- const { ... } = req.body: Lấy dữ liệu từ yêu cầu gửi lên.
- User.findOne({ username }): Tìm xem tên đăng nhập đã bị trùng chưa.
- bcrypt.hash(password, 10): Mã hóa mật khẩu với muối (salt) là 10 vòng.
- User.create(...): Lưu người dùng mới vào database. displayName được kết hợp từ họ và tên.
- res.sendStatus(204): Trả về trạng thái thành công nhưng không có nội dung.

Hàm signIn (Đăng nhập):

- bcrypt.compare(...): So sánh mật khẩu người dùng nhập với mật khẩu đã mã hóa trong DB.
- jwt.sign(...): Tạo Access Token chứa userId.
- crypto.randomBytes(64).toString('hex'): Tạo một chuỗi ngẫu nhiên cực dài làm Refresh Token.
- Session.create(...): Lưu Refresh Token vào DB để quản lý phiên.
- res.cookie(...): Gửi Refresh Token về trình duyệt qua Cookie với tùy chọn httpOnly (tránh bị script bên ngoài đánh cắp).

Hàm signOut (Đăng xuất):

- Session.deleteOne(...): Xóa phiên làm việc trong cơ sở dữ liệu.
- res.clearCookie(...): Xóa sạch cookie ở phía trình duyệt.

2. Kết nối Cơ sở dữ liệu (libs/db.js)

- *mongoose.connect(...):* Sử dụng chuỗi kết nối từ file môi trường (.env) để liên kết tới MongoDB.
- *process.exit(1):* Nếu kết nối thất bại, dừng ngay lập tức chương trình để tránh treo hệ thống.

3. Middleware Bảo vệ (middlewares/authMiddleware.js)

- *req.headers["authorization"]:* Lấy mã token từ Header gửi kèm.
- *token.split(" ")[1]:* Loại bỏ chữ "Bearer" để lấy chuỗi JWT nguyên bản.
- *jwt.verify(...):* Giải mã và kiểm tra tính hợp lệ/hết hạn của token.
- *req.user = user:* Gắn thông tin người dùng vào yêu cầu để các hàm sau có thể sử dụng.
- *next():* Cho phép yêu cầu đi tiếp đến các Route bên trong.

4. Model Phiên làm việc (models/session.js)

- *userId:* Liên kết (ref) tới bảng User.
- *expiredAt:* Thời điểm phiên làm việc hết hiệu lực.

- *index({expiredAt: 1}, {expiredAfterSeconds: 0}):* Đây là tính năng TTL (Time To Live) của MongoDB, tự động xóa bản ghi khi đến thời gian hết hạn.

5. Cấu hình Server chính (server.js)

- app.use(express.json()): Cho phép server đọc dữ liệu định dạng Json.
- app.use(cookieParser()): Cho phép server đọc dữ liệu từ cookie của người dùng.
- app.use('/api/auth', authRoute): Các route công khai (không cần đăng nhập).
- app.use(protectedRoute): Đóng vai trò là bức tường bảo vệ. Tất cả các route nằm bên dưới dòng này đều bắt buộc phải có token hợp lệ mới truy cập được (User, Friends, Messages, Conversations,...).

III. Tổng quan luồng hoạt động của hệ thống

1. Luồng Đăng ký (Sign Up)

- **Gửi yêu cầu:** Client sẽ gửi thông tin đăng ký (email, mật khẩu,...) lên server.
- **Xử lý tại server:** Hệ thống kiểm tra trùng lặp dữ liệu. Mật khẩu được mã hóa một chiều (hashing) bằng bcrypt trước khi lưu vào cơ sở dữ liệu để đảm bảo an toàn.
- **Cấp quyền:** Sau khi tạo tài khoản thành công, server tạo ra một cặp thẻ: **access token** (thời hạn ngắn, trả về cho client lưu tạm trên RAM/State) và **refresh token** (thời hạn dài, lưu vào HttpOnly Cookie và lưu một bản vào Database).

2. Luồng Đăng nhập & Duy trì phiên (Sign In & Silent Refresh)

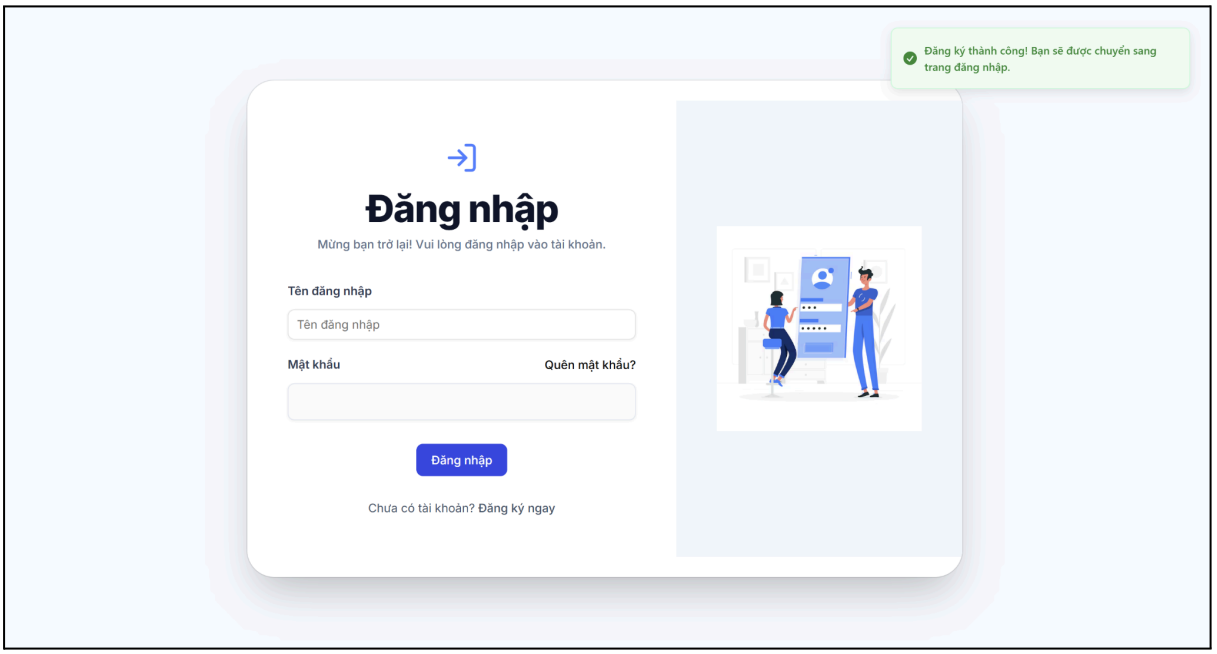
- **Đăng nhập chủ động:** Client gửi thông tin xác thực. Server đối chiếu mật khẩu đã mã hóa. Nếu hợp lệ, server cấp lại cặp token mới (access token và refresh token) tương tự như quá trình đăng ký.
- **Gia hạn Token ngầm (Cơ chế quan trọng):** Khi Access Token hết hạn, các request tiếp theo sẽ bị server từ chối gây ra lỗi 401 Unauthorized.
    - Sử dụng Axios Interceptor ở Frontend sẽ tự động bắt lỗi này, tạm dừng request hiện tại.
    - Client gọi ngầm api /refresh, trình duyệt tự động đính kèm httpOnly cookie chứa refresh token lên Server.
    - server xác thực cookie, nếu hợp lệ sẽ cấp access token mới. Trạm đánh chặn lập tức gắn token mới này vào request đang bị tạm dừng và gửi lại

lên server. Toàn bộ quá trình diễn ra xuyên suốt, người dùng không bị gián đoạn hay phải đăng nhập lại.

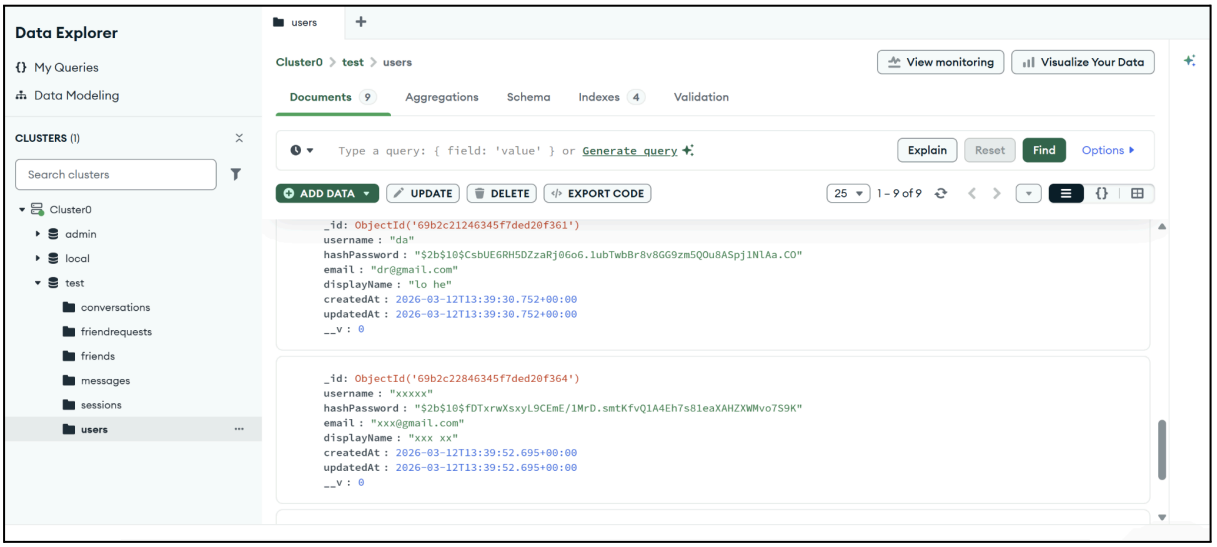
3. Luồng Đăng xuất (Sign Out)

- **Xử lý tại client:** Người dùng nhấn đăng xuất, giao diện gọi api gửi yêu cầu lên server.
- **Thu hồi quyền (Server):** Server tìm và xóa bỏ refresh Token tương ứng trong database (bảng session), đồng thời ra lệnh xóa httpOnly cookie trên trình duyệt của người dùng.
- **Xóa trạng thái (Client):** Kho lưu trữ trạng thái (Zustand) xóa trắng access token và thông tin User, hệ thống tự động điều hướng người dùng quay trở lại màn hình Đăng nhập.

IV. Kết quả



Logout
Đăng nhập thành công!


Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
Cluster0
admin
local
test
conversations
friendrequests
friends
messages
sessions
users
users
Cluster0 > test > users
View monitoring
Visualize Your Data
Documents 9
Aggregations
Schema
Indexes 4
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT CODE
25
1 - 9 of 9
_id: ObjectId('69b2c21246345f7ded20f361')
username : "da"
hashPassword : "$2b$10$CsbUE6RH5DZzaRj06o6.1ubTwbBr8v8GG9zm5QOu8ASpj1NlAa.CO"
email : "dr@gmail.com"
displayName : "lo he"
createdAt : 2026-03-12T13:39:30.752+00:00
updatedAt : 2026-03-12T13:39:30.752+00:00
__v : 0
_id: ObjectId('69b2c22846345f7ded20f364')
username : "xxxxx"
hashPassword : "$2b$10$fDTxrwXsxyL9CEmE/1MrD.smtKfvQ1A4Eh7s81eaXAHZXWMvo7S9K"
email : "xxx@gmail.com"
displayName : "xxx xx"
createdAt : 2026-03-12T13:39:52.695+00:00
updatedAt : 2026-03-12T13:39:52.695+00:00
__v : 0